Table of Contents

INTRODUCTION **2**

Overall workflow (repetition) 2
Frondend Overview 2

INSTALLING JWT-DECODE **5**

ADDING TYPES **5**

ADDING AUTHCONTEXT AND AUTHSERVICE **6**

Understanding the Context design pattern 6
Adding AuthContext 7
Adding AuthService 10

MODIFYING ITEMSERVICE **12**

ADDING/MODIFYING UI COMPONENTS **14**

Adding AuthSection 14
Adding RegisterPage 15
Adding LoginPage 18
Hiding UI Elements from Unauthorized Users 20
ItemListPage.tsx *20*
For ItemTable.tsx *22*
Modify ItemGrid.tsx *23*

SETTING UP THE ROUTING **24**

Adding ProtectedRoute.tsx 24
Modifying NavMenu.tsx 25
Modifying App.tsx 26

TEST THE APP **28**

Demo: JWT Authentication-frontend

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

This demo will continue the JWT authentication topic by setting up a secure frontend with JWT (JSON Web Token) authentication using React.

Overall workflow (repetition)

JWT (JSON Web Token) is a popular authentication technique for frontend-backend communication. The core concept is that each authenticated user receives a unique JWT token from the backend, which must be included in requests to access protected resources.

User Registration

- User submits registration form through frontend
- Backend validates and stores user credentials in database
- No token is generated during registration

User Login

- User submits login credentials through frontend
- Backend verifies credentials against database. If valid, backend generates a signed JWT token containing user information
- Frontend receives and stores the JWT token (typically in **localStorage**)

Accessing Protected Routes

- User attempts to access protected resources (usually Create/Update/Delete operations)
- Frontend automatically includes JWT token in the Authorization header of API requests
- Backend validates the JWT token's signature and expiration
    - *Valid token*: Request proceeds to the protected endpoint
    - *Invalid/expired token*: Request rejected with 401 Unauthorized

User Logout

- Frontend removes the JWT token from **localStorage**
- Backend
    - No backend communication required for JWT (stateless nature of JWT)
    - Note: SignOutAsync() is used for cookie-based authentication, not JWT. Key Point: Unlike session-based authentication, JWT is stateless. The backend *doesn't store active sessions*, making it ideal for scalable web APIs.

Frondend Overview

To implement the workflow, we will need to create and modify several frontend files:

- **UI Components:** Create dedicated components for the **registration** and **login** forms (RegisterPage.tsx, LoginPage.tsx). Also, modify UI elements like the navigation bar (NavMenu.tsx) to display the user's authentication status, and conditionally show or hide buttons for protected actions (ItemListPage.tsx, ItemTable.tsx, ItemGrid.tsx).

- **Authentication Services:** Create a central authentication service (AuthService.ts) to handle all API calls for user authentication. We also need to update the ItemService.ts to include the **JWT token** in the header of requests for protected routes (create, update, delete).
- **State Management & Routing:** Use React's Context design pattern (AuthContext.tsx) to manage the authentication state. You will also use a **ProtectedRoute** component to restrict access to certain routes and update the main application file (App.tsx) to set up the routing structure.

Below is the list with concise explanation for the **Frontend (React).**

Data Models & Types

- myshop/src/types/auth.ts & myshop/src/types/user.ts: Define the data shapes for authentication (e.g., LoginDto, RegisterDto) and user information. This is a best practice in TypeScript for improved code quality and error prevention.

Core Authentication Logic

- myshop/src/auth/AuthContext.tsx: Manages the global authentication state using the **provider-consumer pattern**. It holds the JWT token and user data, making them available throughout the application.
- myshop/src/auth/AuthService.ts: A central service that handles all backend API calls for authentication, such as login and register. It sends data to the backend and handles the response.

UI Components

- myshop/src/auth/LoginPage.tsx & RegisterPage.tsx: These UI components contain the forms where users enter their credentials.
- myshop/src/auth/AuthSection.tsx: A UI component that dynamically shows "Login/Register" links or a "Logout" button based on the user's authentication status.
- myshop/src/items/ItemListPage.tsx, ItemTable.tsx, ItemGrid.tsx: UI components modified to conditionally hide the create/update/delete buttons from unauthorised users.

Route Protection & Integration

- myshop/src/auth/ProtectedRoute.tsx: A special component that acts as a **gatekeeper** for your application's private routes. It only allows access to a route if the user is authenticated.
- myshop/src/App.tsx: The main application file that ties everything together. It wraps the entire application in AuthProvider and sets up the routing, including the ProtectedRoute for private routes.

Service Integration

- myshop/src/items/ItemService.ts: This is an example of a data service that requires authentication. It includes the **JWT token** in the Authorization header for requests to protected API endpoints.

Note on Security and Storage Choice

In this tutorial, we store the JWT in localStorage for simplicity and ease of understanding. This is a common pattern in learning environments because it allows the React frontend to manage authentication independently without complex server-side setup.

Production Best Practice: While storing tokens in localStorage works well for demos, it is vulnerable to **Cross-Site Scripting (XSS)** attacks because any client-side JavaScript can read localStorage.
In production applications, the modern security standard is to store short-lived access tokens **in-memory** (inside React state) and use **HTTP-Only, SameSite cookies** issued by the backend for session persistence. Because JavaScript cannot access HTTP-Only cookies, this pattern protects user tokens from being stolen via XSS (for yourself to explore, the learn to learn skill).

Installing jwt-decode

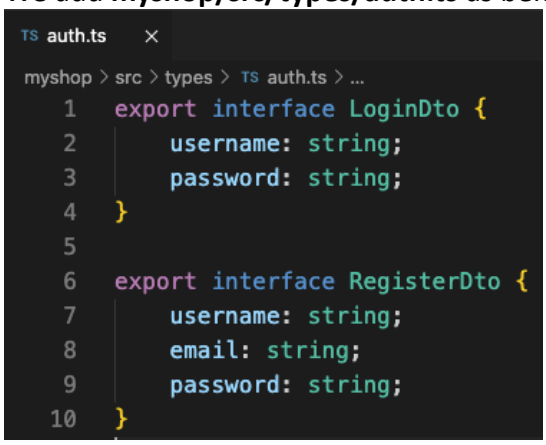
We start with installing a dependency jwt-decode by running the commands:

```
cd myshop
npm install jwt-decode
```

`jwt-decode`: This library is essential for client-side decoding of the JSON Web Token (JWT) to access user information without needing another API call.

Adding types

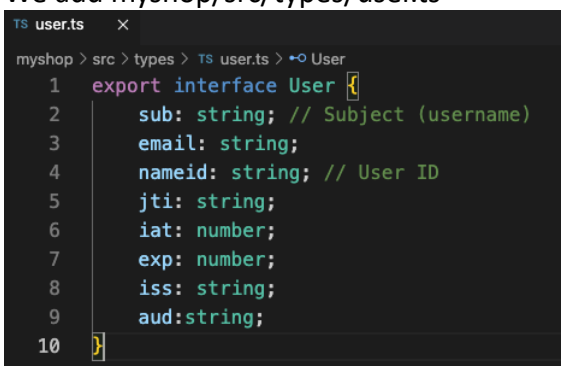
We add **myshop/src/types/auth.ts** as below



This code defines the structure for data used in user authentication. auth.ts contains two TypeScript interfaces: LoginDto and RegisterDto.

- **LoginDto**: This interface specifies the data needed to log in a user. It requires a **username** and a **password**, which are the standard fields for user authentication.
- **RegisterDto**: This interface specifies the data needed to create a new user account. It includes the login fields (**username, email, password**). You could optionally add more required information such as **firstName**, and **lastName**.

We add myshop/src/types/user.ts



This TypeScript interface, User, defines the structure for a decoded JSON Web Token (JWT). It specifies the various claims (pieces of information) contained within the token, which provide details about the authenticated user and the token itself.
Here's a brief explanation of some of the properties:

- **sub** (Subject): The principal (user) that the token is about. It's the **username**.**nameid** (Name ID): A unique identifier for the user, typically the **User ID** from the database.

- **jti** (JWT ID): A unique identifier for the JWT, which can be used to prevent replay attacks.
- **iat** (Issued At): A timestamp indicating when the token was issued.
- **exp** (Expiration Time): A timestamp indicating when the token will expire.
- **iss** (Issuer): Identifies the authority that issued the token.
- **aud** (Audience): Identifies the recipients that the token is intended for.

Adding AuthContext and AuthService

Understanding the Context design pattern

We add two files **myshop/src/auth/AuthContext.tsx** and **myshop/src/auth/AuthService.ts**. These two files are essential.

- **AuthService.ts: This file contains all the logic for communicating with the backend's authentication endpoints. It handles the fetch calls for logging in and registering users and is responsible for storing the returned JSON Web Token (JWT).**
- **AuthContext.tsx: This file manages the application's global authentication state. It stores the JWT and decoded user information, making it accessible to all other components.**

There are fundamentally two main states we need to manage and distinguish throughout the app's lifecycle: **authenticated (logged in)** and **unauthenticated (logged out)**.

- **Unauthenticated State**: The user has not been identified. The app only allows access to public content like login and registration pages. Any attempt to access protected resources is blocked.
- **Authenticated State**: The user has successfully logged in, and the application has stored their JWT. The app now grants access to protected routes and data. The user interface also changes to reflect their logged-in status.

The Context Design Pattern

We use the **React Context** design pattern to manage the application's authentication state. This pattern solves a common problem called "**prop drilling**," where you have to manually pass data through many layers of nested components to reach a deeply-nested child. Instead, Context creates a central data store, making the data accessible to any component that needs it without explicit, manual passing.

The AuthContext implements this pattern by creating a **provider-consumer relationship**.

- **The Provider**

The **AuthContext.Provider** is the **single source of truth** for your authentication state. It is a component that wraps the parts of your application that need access to the user's login status. Within this provider, we use React hooks like **useState** and **useEffect** to manage the login/logout state and the user data.

- **The Consumer**

Any component that needs to access the authentication data is considered a **consumer**. While older versions of React used a dedicated <Context.Consumer> component, modern React accomplishes this with the **useContext** hook. This hook allows a component to

"consume" the data from the nearest AuthContext.Provider above it in the component tree. Components that act as consumers include:

- **UI Components:** A navigation bar, for example, can consume the authentication state to dynamically show a "Login" button or a "Logout" button.
- **Data Services:** A service that fetches data from a protected API can consume the authentication token from the context and include it in the request headers, ensuring the request is authorised, such as the create/update/delete functions of item.

Adding AuthContext

First create a folder auth under myshop/src, then add AuthContext.tsx:

AuthContext.tsx

6-React-Intro > Demo-react-10-authentication-frontend > myshop > src > auth > AuthContext.tsx > AuthProvider > login

```tsx
import { createContext, useState, useContext, useEffect, useCallback, ReactNode } from 'react';
import { jwtDecode } from 'jwt-decode';
import { User } from '../types/user';
import { LoginDto } from '../types/auth';
import * as authService from './AuthService';

interface AuthContextType { // Define the shape of the auth context
    user: User | null;
    token: string | null;
    login: (credentials: LoginDto) => Promise<void>;
    logout: () => void;
    isLoading: boolean;
}

interface AuthProviderProps {
    children: ReactNode;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: AuthProviderProps) {
    const [user, setUser] = useState<User | null>(null);
    const [token, setToken] = useState<string | null>(() => localStorage.getItem('token'));
    const [isLoading, setIsLoading] = useState<boolean>(true);

    useEffect(() => { // Check token validity on mount and when token changes ⋯
    }, [token]);

    const login = async (credentials: LoginDto) => { // Login and store token ⋯
    };

    const logout = () => { // Clear token and user data ⋯
    };

    return ( // Provide context to children
    <AuthContext.Provider value={{ user, token, login, logout, isLoading }}>
        {children}
    </AuthContext.Provider>
    );
}

export function useAuth(): AuthContextType { // Custom hook to use auth context
    const context = useContext(AuthContext);
    if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
    }
    return context;
}
```

import: This section imports all the necessary tools and types.

- **React hooks (useState, useEffect)**: Used to manage the state of the component.
- **Context API (createContext, useContext)**: The core of this file, used to create and access the global authentication state.
- **jwtDecode**: A library to read data from the JWT.
- **Interfaces (User, LoginDto)**: Imports the data structures for user and login information.
- **authService**: Imports the functions that handle the actual login API calls to the backend.

interface AuthContextType: This defines the **shape** of the data that will be available through the context. It specifies what other components can access: the user object, the token, and functions to login and logout.
interface AuthProviderProps: The data interface for AuthProvider.
const AuthContext = createContext(...): This creates the context itself. It's the object that will hold and distribute the authentication state.
export const AuthProvider: React.FC: This is the main component that manages the authentication state. It "provides" the context to all auth consumers.

- **useState**: These lines create state variables to hold the **user data,** the **JWT (token),** and a loading status (**isLoading**). The token is initially read from **localStorage** to handle persistent login sessions.
- **useEffect**: This React hook runs when the component first loads and whenever the token variable changes.

const login = async (...): This function handles the login process.
const logout = () => { ... }: This function handles the logout process.
return (...): This line returns the **AuthContext.Provider**. This makes the value prop (containing the user, token, and login/logout functions) available to any auth consumers.
export const useAuth = () => { ... }: This is a **custom hook**. It provides shortcut for any other component that are consumers of the authentication context. Instead of directly using **useContext(AuthContext),** a developer can simply call the shortcut **useAuth()** to get all the data and functions they need. This makes the code cleaner and less prone to errors.

UseEffect

```
26      useEffect(() => { // Check token validity on mount and when token changes
27      if (!token) {
28          setIsLoading(false);
29          return;
30      }
31      try {
32          const decodedUser: User = jwtDecode(token);
33          if (decodedUser.exp * 1000 > Date.now()) { // Check if the token is expired
34          setUser(decodedUser);
35          } else { // Token is expired, clear it
36          console.warn("Token expired");
37          localStorage.removeItem('token');
38          setUser(null);
39          setToken(null);
40          }
41      } catch (error) {
42          localStorage.removeItem('token');
43          setUser(null);
44          setToken(null);
45      } finally {
46          setIsLoading(false);
47      }
48      }, [token]);
```

This useEffect hook's primary job is to **validate the user's authentication token** every time the component loads or the token state changes.

1. **Check for a Token**: The code first checks if a token exists in the state. If a token is found, it proceeds; otherwise, it does nothing and moves on.
2. **Decode and Validate**: It attempts to decode the token to get the expiration date (exp). It then checks if the token's expiration time is still in the future.
    - If **valid**, it sets the user state to the decoded information, confirming the user is logged in.
    - If **expired or invalid**, it logs a warning, removes the invalid token from local storage, and clears the user and token states.
3. **Handle Errors**: The try...catch block ensures that if the token is corrupted or unreadable, the application handles the error gracefully by logging it and clearing the token.
4. **Signal Readiness**: Finally, it sets the **isLoading** state to false. This signals to the rest of the application that the initial authentication check is complete and it's safe to render the correct content (either public or protected).

Login and Logout

```
50    const login = async (credentials: LoginDto) => { // Login and store token
51        const { token } = await authService.login(credentials);
52        localStorage.setItem('token', token);
53        const decodedUser: User = jwtDecode(token);
54        // console.log(token); // Debugging line to check the token only in development, otherwise
          security breach in production
55        setUser(decodedUser);
56        setToken(token);
57    };
58
59    const logout = () => { // Clear token and user data
60        localStorage.removeItem('token');
61        setUser(null);
62        setToken(null);
63    };
```

The login function is an asynchronous operation that logs a user into the application.

1. It calls the **authService.login()** function, passing it the user's credentials. This function sends the login request to the backend and returns the authentication token.
2. Upon a successful response, it saves the received **token** to the browser's **localStorage** to persist the user's session.
3. The token is then immediately decoded using **jwtDecode()** to extract the user's information. **The console.log is not secure, only for debugging purpose. It should never be used in production code.** Why It's Not Secure?
    - Exposure of Sensitive Data: A JWT is a sensitive piece of data that proves a user's identity. Anyone with physical access to the computer, or even those on the same network if the connection isn't fully secure, could potentially view this token.
    - Security Vulnerability: If a malicious script were to be injected into the page via a cross-site scripting (XSS) attack, it could read the token directly from the console output.
4. Finally, it updates the component's state by setting the user and token variables, which makes the logged-in status available to the rest of the application.

logout function

The logout function is a simple, synchronous operation that clears the user's session.

1. It removes the token from **localStorage**.
2. It resets the component's state by setting both the user and token variables to null. This instantly changes the application's state from authenticated to unauthenticated.

Adding AuthService

Add AuthService.ts under myshop/src/auth

AuthService.ts

6-React-Intro > Demo-react-10-authentication-frontend > myshop > src > auth > AuthService.ts > register

```ts
import { LoginDto, RegisterDto } from '../types/auth';

const API_URL = import.meta.env.VITE_API_URL;

export interface AuthResponse {
    token: string;
}

export interface RegisterResponse {
    message?: string;
    [key: string]: unknown;
}

export const login = async (credentials: LoginDto): Promise<AuthResponse> => {
    const response = await fetch(`${API_URL}/api/Auth/login`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify(credentials),
    });

    if (!response.ok) { // Attempt to extract backend error message if available
        const errorData = await response.json().catch(() => null);
        throw new Error(errorData?.message || 'Login failed. Please check your credentials.');
    }
    console.log(response); // only in development
    return response.json();
};

export const register = async (userData: RegisterDto): Promise<RegisterResponse> => {
    const response = await fetch(`${API_URL}/api/Auth/register`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify(userData),
    });

    if (!response.ok) {
        const errorData = await response.json().catch(() => null);
        // Handle array of ASP.NET Identity errors: [{ description: "..." }]
        if (Array.isArray(errorData)) {
            const errorMessages = errorData.map((err) => err.description).join(', ');
            throw new Error(errorMessages || 'Registration failed');
        }
        // Handle standard API problem details or generic messages
        throw new Error(errorData?.title || errorData?.message || 'Registration failed');
    }

    return response.json();
};
```

This code is a **service file** that handles all API calls related to user authentication for the frontend.
It contains two asynchronous functions:
login: This function sends a POST request to the backend's **/api/Auth/login** endpoint with the user's login credentials. If the backend responds with a success status, it returns the JWT token from the response. If the login fails, it throws an error.

register: This function sends a POST request to the backend's **/api/Auth/register** endpoint with the user's registration data.

- If the backend responds with a success status, it returns the confirmation.
- **if (!response.ok)**: If the status code is not in the 200 range (e.g., 400 Bad Request), the code proceeds to handle the error.
    - **await response.json()**: The backend returns a JSON object containing an **array of error messages** from the **UserManager** service. There are multiple error descriptions because the backend's validation process can find more than one issue with the data submitted by the user, such as a username being too short and a password not meeting complexity requirements.
    - **errorData.map(...)**: This is a key step. The map method iterates through the array of errors returned by the backend. For each error object in the array, it extracts just the description property, which contains the user-friendly error string.
    - **.join(', ')**: The join method then combines all the extracted error descriptions from the map operation into a single, comma-separated string. This makes it easy to display a concise and informative error message to the user, like "Username is already taken, Password must contain a number."

Modifying ItemService

Since the backend **ItemAPIController.cs** now uses the **[Authorize]** attribute to protect its create, update, and delete actions, the frontend must include a valid **JWT token** with every request to these endpoints.
To achieve this, we need to modify the **myshop/src/items/ItemService.tsx** file to include the token in the Authorization header of each fetch call.
The correct and professional way to do this is to **add a header** in the format `Authorization: Bearer <your_jwt_token>` , and **use the header in all fetch requests** that access the protected API endpoints.

In the **ItemService.tsx** file, we first change the header:

Before:

```
3   const headers = {
4     'Content-Type': 'application/json',
5   };
```

After:

```
3   // Helper function to get headers, including the auth token if it exists
4   const getAuthHeaders = () => {
5     const token = localStorage.getItem('token');
6     const headers: HeadersInit = {
7       'Content-Type': 'application/json',
8     };
9     if (token) {
10      headers['Authorization'] = `Bearer ${token}`;
11    }
12    return headers;
13  };
```

This is a **helper function** that creates and returns the correct **HTTP headers** for an authenticated API request. It's designed to be reused across all your fetch calls that require a JWT token. Here's how it works:

1. It retrieves the **JWT token** from the browser's local storage.
2. It initialises a headers object with the standard `Content-Type: application/json`.
3. It checks if a token exists. If it does, it adds the **Authorization header** with the token prefixed by `Bearer` . This is the standard format required by your backend's JWT authentication middleware.
4. Finally, it returns the complete headers object, ready to be used in your fetch requests.

Then we modify the fetch requests of create/update/delete,

```
33  export async function createItem(item: Item): Promise<Item> {
34    const response = await fetch(`${API_URL}/api/itemapi/create`, {
35      method: 'POST',
36      headers: getAuthHeaders(), // Use the new helper here
37      body: JSON.stringify(item),
38    });
39    return handleResponse<Item>(response);
40  }
41
42  export async function updateItem(item: Item): Promise<Item> {
43    const response = await fetch(`${API_URL}/api/itemapi/update/${item.itemId}`, {
44      method: 'PUT',
45      headers: getAuthHeaders(), // Use the new helper here
46      body: JSON.stringify(item),
47    });
48    return handleResponse<Item>(response);
49  }
50
51  export async function deleteItem(itemId: number): Promise<void> {
52    const response = await fetch(`${API_URL}/api/itemapi/delete/${itemId}`, {
53      method: 'DELETE',
54      headers: getAuthHeaders(), // Use the new helper here
55    });
56    if (!response.ok) {
57      throw new Error(`Failed to delete item (status ${response.status})`);
58    }
```

Adding/modifying UI Components

Now we start adding some UI components, including the AuthSection that appears on the top right of the webapp to indicate the current login/logout status, and the Registration and Login form.

Adding AuthSection

Add myshop/src/auth/AuthSection.tsx as below

AuthSection.tsx

6-React-Intro > Demo-react-10-authentication-frontend > myshop > src > auth > AuthSection.tsx > ...

```tsx
import { Link } from 'react-router-dom';
import { useAuth } from './AuthContext';
import { Nav, Dropdown } from 'react-bootstrap';

export default function AuthSection() {
    const { user, logout } = useAuth();

    return (
        <Nav>
            {user ? (
                <Dropdown align="end">
                    <Dropdown.Toggle as={Nav.Link} id="dropdown-user">
                        Welcome, {user.sub}
                    </Dropdown.Toggle>
                    <Dropdown.Menu>
                        <Dropdown.Item onClick={logout}>Logout</Dropdown.Item>
                    </Dropdown.Menu>
                </Dropdown>
            ) : (
                <>
                    <Nav.Link as={Link} to="/login">Login</Nav.Link>
                    <Nav.Link as={Link} to="/register">Register</Nav.Link>
                </>
            )}
        </Nav>
    );
};
```

AuthSection.tsx, is a navigation element that dynamically changes based on a user's authentication status.

1. It uses the **useAuth()** custom hook to access the authentication state, which includes the current **user** object and the **logout** function. The **useAuth()** hook is a professional shortcut that internally calls **useContext(AuthContext)** to get this data. It checks if a user is authenticated (user is not null).
2. **If a user is logged in**, it displays a Dropdown menu with a personalised welcome message (Welcome, {user.sub}) and a "**Logout**" button.
3. **If no user is logged in**, it displays two **Nav.Link** components for navigating to the "**Login**" and "**Register**" pages.

Adding RegisterPage

Adding myshop/src/auth/RegisterPage.tsx as below



This creates a **user registration form**. It handles the entire process on the client side, from capturing user input to submitting the data and handling the response from the API.

State Management

- **useState hooks**: The component uses three useState hooks to manage its state:
    1. **formData**: Stores the user's input for username, email, and password.
    2. **error**: Stores any error messages received from the backend.
    3. **success**: Stores a success message after a successful registration.
    4. `isSubmitting`**:** A boolean state tracking whether an API request is currently in progress, preventing duplicate form submissions and enabling loading feedback.
- **useNavigate hook**: This hook from react-router-dom is used to programmatically navigate the user to the login page after a successful registration.
- **useEffect hook:** Automatically triggers when the success state updates. It manages a 2-second delay before redirecting to /login and returns a cleanup function (clearTimeout) to prevent memory leaks if the component unmounts before the timer finishes.

Form Logic

- **handleChange**: This function is called every time a user types in an input field. It updates the **formData** state to reflect the latest user input.
- **handleSubmit:** This asynchronous function handles the form submission (React.SubmitEvent):
    - It prevents the default browser page reload (e.preventDefault()).
    - It sets isSubmitting to true to disable form controls and show a loading spinner.
    - It calls authService.register(formData) to send the user data to the API.
    - **On Success:** Sets a success message, triggering the useEffect redirect workflow.
    - **On Error:** Catches any API exceptions, extracts the error message, and sets the error state.
    - **Cleanup (finally):** Resets isSubmitting back to false when the operation finishes.

Return Form

```jsx
    return (
        <Container className="mt-5" style={{ maxWidth: '500px' }}>
            <h2>Register</h2>
            {error && <Alert variant="danger">{error}</Alert>}
            {success && <Alert variant="success">{success}</Alert>}

            <Form onSubmit={handleSubmit}>
                <Form.Group className="mb-3">
                    <Form.Label>Username</Form.Label>
                    <Form.Control type="text" name="username" value={formData.username} onChange=
                    {handleChange} disabled={isSubmitting || !!success} required />
                </Form.Group>

                <Form.Group className="mb-3">
                    <Form.Label>Email</Form.Label>
                    <Form.Control type="email" name="email" value={formData.email} onChange=
                    {handleChange} disabled={isSubmitting || !!success} required />
                </Form.Group>

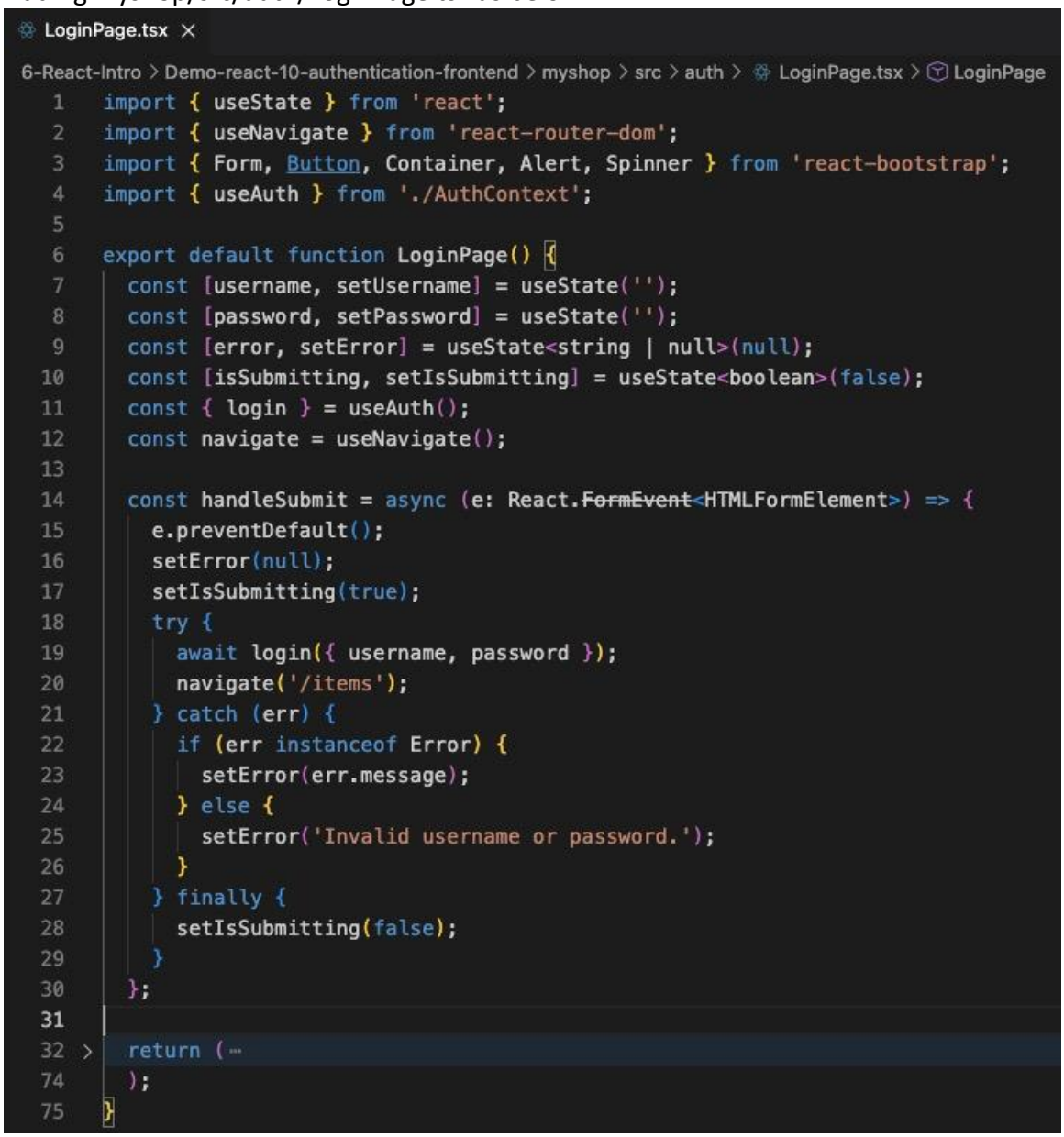
                <Form.Group className="mb-3">
                    <Form.Label>Password</Form.Label>
                    <Form.Control type="password" name="password" value={formData.password}
                    onChange={handleChange} disabled={isSubmitting || !!success} required />
                </Form.Group>

                <Button variant="primary" type="submit" disabled={isSubmitting || !!success}>
                    {isSubmitting ? (
                        <>
                        <Spinner animation="border" size="sm" className="me-2" />
                        Registering...
                        </>
                    ) : (
                        'Register'
                    )}
                </Button>
            </Form>
        </Container>
    );
```

- **Return Statement:** Renders the registration form centered with a max-width container using React Bootstrap components (Container, Form, Button, Alert, Spinner).
- **Conditional Alerts:** Conditionally displays an Alert component with either an error or success message based on the current state.
- **Form Protection & Loading Feedback:** All form inputs and the submit button are automatically disabled when isSubmitting is true or after a success state is reached. The submit button dynamically displays an animated React Bootstrap Spinner while the API request is processing.

Adding LoginPage

Adding myshop/src/auth/LoginPage.tsx as below



```tsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Form, Button, Container, Alert, Spinner } from 'react-bootstrap';
import { useAuth } from './AuthContext';

export default function LoginPage() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [isSubmitting, setIsSubmitting] = useState<boolean>(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    setError(null);
    setIsSubmitting(true);
    try {
      await login({ username, password });
      navigate('/items');
    } catch (err) {
      if (err instanceof Error) {
        setError(err.message);
      } else {
        setError('Invalid username or password.');
      }
    } finally {
      setIsSubmitting(false);
    }
  };

  return (…
  );
}
```

This code creates a **user login form**. It handles the entire login process, from capturing user input to authenticating with the API and managing the local application state.

State Management and Hooks

- **useState hooks**: The component uses three **useState** hooks to manage its state: username, password, and error. These hold the user's input and any login error message. isSubmitting**:** Tracks whether the login request is currently processing to handle UI loading states.
- **useAuth hook**: This custom hook provides access to the login function from your **AuthContext**, which handles the actual communication with the backend.
- **useNavigate hook**: This hook is used to redirect the user to a new page (e.g., /items) upon a successful login.

Form Logic

- **handleSubmit**: This asynchronous function is triggered when the form is submitted.

- **e.preventDefault():** Prevents the browser from reloading the page, which is essential for a single-page application.
- Sets `isSubmitting` to `true` to lock input fields and display a button spinner.
- **try...catch block:** It attempts to call the login function from **AuthContext** with the user's credentials. If the backend authentication is successful, the try block executes, and the user is redirected. If the login fails, the catch block executes, a generic error message is set, and the error is logged to the console.

UI Rendering

```
32    return (
33      <Container className="mt-5" style={{ maxWidth: '500px' }}>
34        <h2>Login</h2>
35        {error && <Alert variant="danger">{error}</Alert>}
36
37        <Form onSubmit={handleSubmit}>
38          <Form.Group className="mb-3" controlId="formBasicUsername">
39            <Form.Label>Username</Form.Label>
40            <Form.Control
41              type="text"
42              placeholder="Enter username"
43              value={username}
44              onChange={(e) => setUsername(e.target.value)}
45              disabled={isSubmitting}
46              required
47            />
48          </Form.Group>
49
50          <Form.Group className="mb-3" controlId="formBasicPassword">
51            <Form.Label>Password</Form.Label>
52            <Form.Control
53              type="password"
54              placeholder="Password"
55              value={password}
56              onChange={(e) => setPassword(e.target.value)}
57              disabled={isSubmitting}
58              required
59            />
60          </Form.Group>
61
62          <Button variant="primary" type="submit" disabled={isSubmitting}>
63            {isSubmitting ? (
64              <>
65                <Spinner animation="border" size="sm" className="me-2" />
66                Logging in...
67              </>
68            ) : (
69              'Login'
70            )}
71          </Button>
72        </Form>
73      </Container>
74    );
```

- **return statement**: This section renders the login form using React Bootstrap components.
- **Conditional Rendering**: The line `{error && <Alert ... />}` conditionally displays an **Alert** component with an error message only if the error state is not null. It provides a clean way to give feedback to the user when login fails.
- **Form Protection:** Disables input controls and the submit button while `isSubmitting` is `true`.

Hiding UI Elements from Unauthorized Users

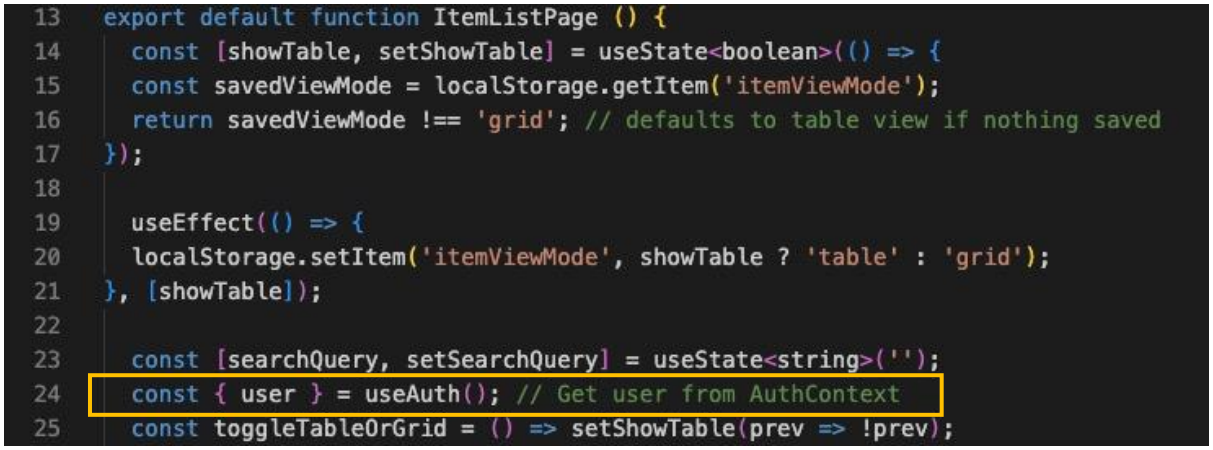
Hiding protected routes and actions from unauthorised users is a professional and secure practice. It improves user experience and security by preventing users from attempting to access functions they don't have permission for.
To implement this, we will modify the following UI components to conditionally hide the create, update, and delete buttons:

- **ItemListPage.tsx**: The main page for displaying items.
- **ItemTable.tsx**: A component that renders items in a table format.
- **ItemGrid.tsx**: A component that renders items in a grid format.

ItemListPage.tsx

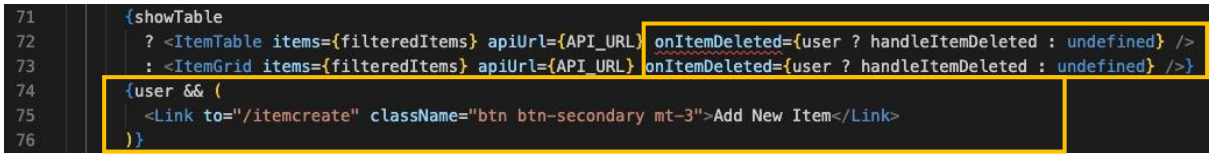
For ItemListPage.tsx, first add import useAuth.





The addition of the **useAuth hook** is to access the current user's authentication status, which will be used to conditionally render UI elements, such as the create/update/delete buttons.

- **import { useAuth } from '../auth/AuthContext';**: This line imports the useAuth hook.
- **const { user } = useAuth();**: This line calls the hook and retrieves the user object from the **authentication context**.



1. The "Add New Item" Button

The Add New Item button is now conditionally rendered using the expression `{user && (...) }`. This means the button will only be displayed on the page if a user object exists, which indicates that the user is authenticated.

2. The onItemDeleted Prop

The onItemDeleted prop passed to both ItemTable and ItemGrid is now a conditional expression: user ? handleItemDeleted : undefined.

- **If user is authenticated**, the handleItemDeleted function is passed as a prop, allowing a logged-in user to delete an item.
- **If user is not authenticated**, undefined is passed instead. This prevents the delete functionality from being available to unauthorised users.

Now we see the red underline because onItemDeleted cannot be undefined for ItemTable and ItemGrid. We now modify the two UI components to allow them to be undefined.

For ItemTable.tsx

Change: the ? after onItemDeleted

```tsx
interface ItemTableProps {
  items: Item[];
  apiUrl: string;
  onItemDeleted?: (itemId: number) => void;
}
```

Change:

```tsx
<td className="text-center">
  {onItemDeleted && (
    <>
      <Link to={`/itemupdate/${item.itemId}`}>Update</Link>
      <Button
        onClick={() => onItemDeleted(item.itemId!)}
        variant="link"
        className="text-danger"
      >Delete</Button>
    </>
  )}
</td>
```

The code was changed to **conditionally render** the "Update" and "Delete" links based on whether the onItemDeleted prop exists.

- **Before:** The "Update" and "Delete" links were always visible to all users.
- **After:** The `{onItemDeleted && (...) }` expression checks for the presence of the onItemDeleted prop. Since this prop is passed to the component only for authenticated users, the links will now only be displayed to users who are logged in.

This prevents unauthorized users from seeing or attempting to access the update and delete functionality, creating a cleaner and more secure UI.

Modify ItemGrid.tsx

Change: the ? after onItemDeleted

```tsx
interface ItemGridProps {
  items: Item[];
  apiUrl: string;
  onItemDeleted?: (itemId: number) => void;
}
```

Change:

```tsx
<div className="mt-auto d-flex justify-content-between gap-2">
  {onItemDeleted && (
    <>
      <Link to={`/itemupdate/${item.itemId}`} className="btn btn-primary">Update</Link>
      <Button onClick={() => onItemDeleted(item.itemId!)} variant="danger">Delete</Button>
    </>
  )}
</div>
```

The ItemGrid.tsx component is updated to **conditionally render** the "Update" and "Delete" buttons and to make the onItemDeleted prop **optional**.

- **Optional Prop**: The **onItemDeleted** prop in the **ItemGridProps** interface was changed to `onItemDeleted?: (itemId: number) => void;`. The question mark (?) makes this prop optional, meaning the ItemGrid component can now be used without a delete function being passed to it. This is crucial for when an unauthenticated user views the grid.
- **Conditional Rendering**: The buttons themselves are now wrapped in a conditional rendering expression: `{onItemDeleted && (...) }`. This check ensures that the buttons only appear if the **onItemDeleted** prop has been provided.

These changes work together to hide the update and delete buttons from unauthorized users, providing a secure and clean UI.

Setting up the routing

Now that we have the frontend logic and UI components for authentication, the final step is to set up the **application routing**. This ensures that the entire app has access to the **AuthContext** and that certain routes are correctly protected.
The core function of AuthContext is to manage the user's authentication states and make it available to all "consumer" components throughout the application.
To accomplish this, we need to create or modify three files:

- **ProtectedRoute.tsx**: This is a component that acts as a **gatekeeper**. It checks if the user is authenticated by **consuming** the data from the `AuthContext` via the `useAuth` hook. This is a key part of the provider-consumer relationship: the provider gives the authentication status, and the `ProtectedRoute` consumes it to make a routing decision. The protected routes are typically those for creating, updating, or deleting data. If a user is not authenticated, the ProtectedRoute component will prevent access and redirect them to a login page.
- **App.tsx**: As the root of your application, its main job is to **provide** the authentication state to all its child components.
    - You will wrap your entire application's routing structure with the `AuthContext.Provider`.
    - This single action makes the user's login status (the provided data) available to every component within the application tree.
- **NavMenu.tsx**: This component will be updated to display the authentication status. It will use the `AuthSection` to conditionally render different UI elements, such as "**Login**" and "**Register**" links for unauthenticated users or a "**Logout**" button and a welcome message for authenticated users.

Adding ProtectedRoute.tsx

Add myshop/src/auth/ProtectedRoute.tsx as below

ProtectedRoute.tsx

6-React-Intro > Demo-react-10-authentication-frontend > myshop > src > auth > ProtectedRoute.tsx > ...

```tsx
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from './AuthContext';

export default function ProtectedRoute()  {
    const { token } = useAuth();

    if (!token) {
        // Redirect them to the /login page, but save the current location they were
        // trying to go to. This allows us to send them along to that page after they login.
        return <Navigate to="/login" replace />;
    }

    return <Outlet />;
};
```

This component, **ProtectedRoute.tsx**, acts as a **gatekeeper** for your application's private routes. Its job is to check if a user is authenticated before allowing them to view a page.

- It uses the **useAuth()** hook to access the authentication state, here the user's token.
- It then runs a simple check: **if (!token)**.

- If there is **no token**, it means the user is not authenticated. The component then returns a `<Navigate to="/login" replace />` element. This redirects the user to the login page and prevents them from seeing the protected content.
- If a **token exists**, the user is authenticated. The component then returns an `<Outlet />` element, which renders the child component (the actual page they were trying to access).

This pattern is a fundamental part of securing a single-page application (SPA) with client-side routing. While it doesn't render visible elements like buttons or text itself, it is a component whose primary job is to control what other UI components are rendered on the screen.

1. If the user is authenticated, it renders the intended protected page (the Outlet).
2. If the user is not authenticated, it renders a <Navigate> component, which redirects the user to the login page.

Modifying NavMenu.tsx

Modifying **myshop/src/shared/NavMenu.tsx** as blow

NavMenu.tsx U

6-React-Intro > Demo-react-10-authentication-frontend > myshop > src > shared > NavMenu.tsx > ...

```tsx
import { Nav, Navbar } from 'react-bootstrap';
import AuthSection from '../auth/AuthSection';

export default function NavMenu() {
  return (
    <Navbar expand="lg">
      <Navbar.Brand href="/">MyShop</Navbar.Brand>
      <Navbar.Toggle aria-controls="basic-navbar-nav" />
      <Navbar.Collapse id="basic-navbar-nav">
        <Nav className="me-auto">
          <Nav.Link href="/">Home</Nav.Link>
          <Nav.Link href="/items">Items</Nav.Link>
          {/* <NavDropdown title="Dropdown" id="basic-nav-dropdown">
            <NavDropdown.Item href="#action/3.1">Action</NavDropdown.Item>
            <NavDropdown.Item href="#action/3.2">
              Another action
            </NavDropdown.Item>
            <NavDropdown.Item href="#action/3.3">Something</NavDropdown.Item>
            <NavDropdown.Divider />
            <NavDropdown.Item href="#action/3.4">
              Separated link
            </NavDropdown.Item>
          </NavDropdown> */}
        </Nav>
      </Navbar.Collapse>
      <AuthSection />
    </Navbar>
  );
};
```

The change is rather simple, importing AuthSection and add it to the end of the Navbar.
I have also commented out the NavDropdown component which is not in use.

Modifying App.tsx

Modify **myshop/src/App.tsx**.



```tsx
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import Container from 'react-bootstrap/Container'
import HomePage from './home/HomePage'
import ItemListPage from './items/ItemListPage'
import ItemCreatePage from './items/ItemCreatePage'
import ItemUpdatePage from './items/ItemUpdatePage'
import NavMenu from './shared/NavMenu'
import LoginPage from './auth/LoginPage'
import RegisterPage from './auth/RegisterPage'
import ProtectedRoute from './auth/ProtectedRoute'
import { AuthProvider } from './auth/AuthContext'
import './App.css'

const queryClient = new QueryClient() // creates the single instance

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <Router>
          <Container className="mt-4">
            <NavMenu />
            <Routes>
              <Route path="/" element={<HomePage />} />
              <Route path="/items" element={<ItemListPage />} />
              <Route path="/login" element={<LoginPage />} />
              <Route path="/register" element={<RegisterPage />} />

              {/* Protected Routes */}
              <Route element={<ProtectedRoute />}>
                <Route path="/itemcreate" element={<ItemCreatePage />} />
                <Route path="/itemupdate/:itemId" element={<ItemUpdatePage />} />
              </Route>

              <Route path="*" element={<Navigate to="/" replace />} />
            </Routes>
          </Container>
        </Router>
      </AuthProvider>
    </QueryClientProvider>
  )
}
```

We imported the UI components such as LoginPage, RegisterPage, and RegisterPage, and the AuthProvider which is core for managing authentication states.

The new App.tsx file has been significantly updated to integrate the **authentication system** and **protect routes** that require a logged-in user.

1. **AuthProvider**: The entire application is now wrapped in the <AuthProvider> component. This is the **provider** part of the provider-consumer pattern. It makes the authentication state (like user info and the JWT token) available to all components within your app's routing structure.
2. **New Routes**: The <Routes> component now includes new paths for the LoginPage and RegisterPage. This allows users to navigate to the authentication forms.
3. **Protected Routes**: Instead of a simple Route, the itemcreate and itemupdate routes are now nested inside a <ProtectedRoute> component.

- The parent <Route> uses element={<ProtectedRoute />} to act as a gatekeeper.
- The nested <Route> components are the actual protected pages.
- This structure ensures that before a user can access these pages, the ProtectedRoute component will run its logic to check if they're authenticated. If not, it will redirect them to the login page.

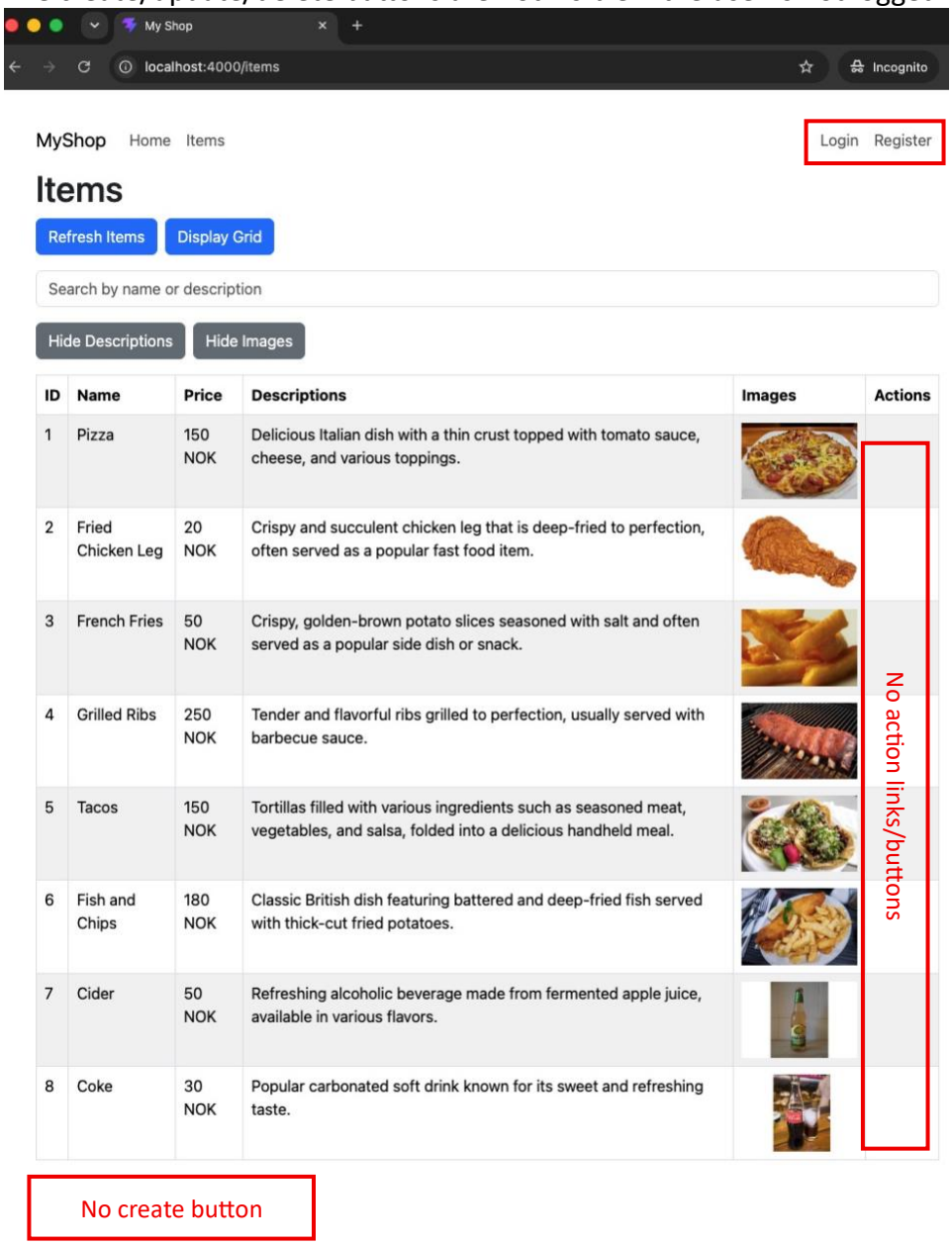
Test the app

Now we can run the app by running:

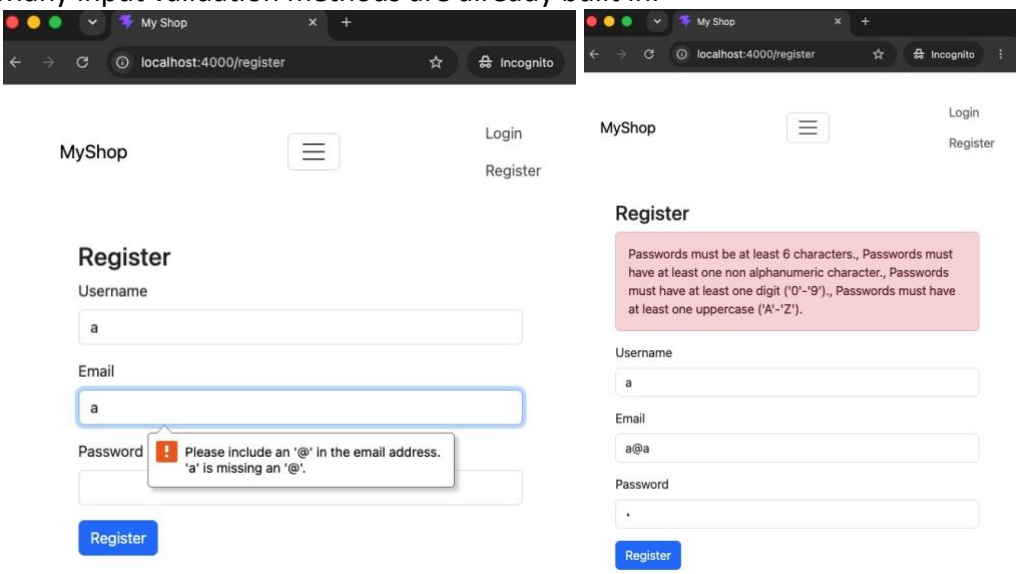
```
npm run dev
```

You can see the AuthSection is displayed on the right of the NavMenu.

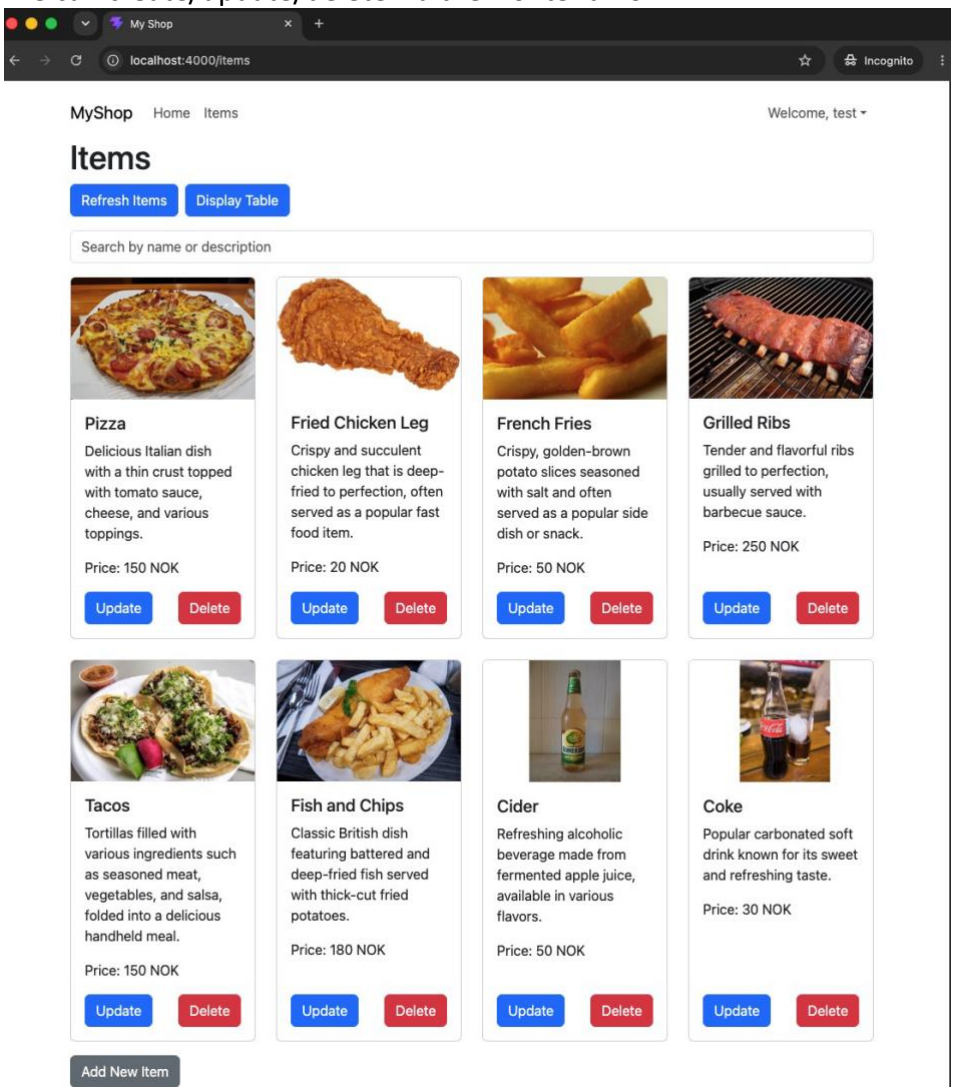
The create/update/delete buttons are not visible if the user is not logged in.



Many input validation methods are already built in.



We can create/update/delete via the frontend now.



Congratulations, you have accomplished the frontend authentication! ☺ Really great achievement!

Congratulation! You have completed all demos of the course! ☺